

# IMPLEMENTAÇÃO 2

Alexandre Niess

Caio Gomes

André Mendes

Rafael Maluf Araujo

14/08/2024

## 1 Introdução

A teoria dos grafos é um campo fundamental da matemática e da ciência da computação, que estuda grafos como uma representação de relações entre objetos. Um grafo completo, também conhecido como grafo  $K_n$ , é um tipo de grafo onde cada par de vértices distintos está conectado por uma aresta única. Este tipo de grafo é um exemplo importante e é frequentemente utilizado em várias aplicações de redes e algoritmos.

Nesta atividade, nosso objetivo é implementar um gerador de todos os subgrafos de um grafo completo com  $n$  vértices, onde o valor de  $n$  é fornecido pelo usuário. Um subgrafo é uma estrutura que pode ser formada a partir de um grafo completo, selecionando um subconjunto de seus vértices e arestas. O número de subgrafos possíveis aumenta exponencialmente com o número de vértices, tornando este um problema interessante e desafiador para a prática de programação e análise de algoritmos.

O programa desenvolvido deve ser capaz de:

- Gerar e listar todos os subgrafos possíveis do grafo completo.
- Informar o número total de subgrafos gerados.

## 2 Função Subgrafos

O professor Silvio nos mostrou uma formula que é capaz de definir o número de subgrafos geráveis através da quantidade de vertices do mesmo, este que uma vez será informado pelo usuário. Uma vez com essas informações, basta apenas implementar a formula:

## 2.1 Estrutura do Código

Listing 1: Implementação da formula

```
void gerarSubgrafos(int n) {  
    // Formula para calcular numero de subgrafos de um grafo completo  
    int numSubgrafos = pow(2, n * (n - 1) / 2);  
    cout << "Numero total de subgrafos gerados: " << numSubgrafos << endl;  
    {...};  
}
```

## 3 Montando Lista de arestas

A parte seguinte do código cria uma lista de todas as arestas possíveis de um grafo completo com  $n$  vértices e armazena essas arestas em um vetor. Para isso, "vector<pair<int, int>> arestas;": Declara um vetor chamado arestas, onde cada elemento é um par de inteiros (pair<int, int>). Cada par representa uma aresta do grafo, onde o primeiro inteiro é o vértice de origem e o segundo inteiro é o vértice de destino.

### 3.1 Estrutura do Código

Listing 2: Implementação da lista de arestas

```
vector<pair<int, int>> arestas;  
  
    // Criando a lista de arestas do grafo completo  
    for (int i = 0; i < n; ++i) {  
        for (int j = i + 1; j < n; ++j) {  
            arestas.push_back({i, j});  
        }  
    }
```

## 4 Gerando subgrafos

A teoria por trás da geração de subgrafos de um grafo completo envolve a combinação de arestas para formar todos os possíveis subgrafos. Um grafo completo é um grafo onde cada par de vértices está conectado por uma aresta única. Dado um grafo completo com  $N$  vértices, o número total de arestas é  $n(n-1)/2$

Sendo assim, para gerar todos os subgrafos devemos:

- Máscara Binária: Utiliza-se uma máscara binária para representar a inclusão ou exclusão de cada aresta em um subgrafo. Cada bit da máscara indica se uma aresta específica está presente no subgrafo.

- Combinações de Arestas: Para cada possível configuração de máscara (de 0 até 2 elevado a k - 1, onde k é o número de arestas), cria-se um subgrafo correspondente com as arestas indicadas pela máscara.
- Total de Subgrafos: O número total de subgrafos possíveis é 2 elevado a k, onde k é o número de arestas no grafo completo.

## 4.1 Estrutura do Código

Listing 3: Implementação do gerador de grafos

```
// Gerando subgrafos
for (int mask = 0; mask < numSubgrafos; ++mask) {
    cout << "Subgrafo-" << mask + 1 << ":-";
    for (int i = 0; i < arestas.size(); ++i) {
        if (mask & (1 << i)) {
            cout << "(" << arestas[i].first << ",-" << arestas[i].second <<
        }
    }
    cout << endl;
}
```

## 5 Main

A main onde será digitado o número de vértices que o usuário deseje.

obs: irá dar "erro" caso o valor inserido seja menor do que 1, isso pois no contexto de grafos, um número de vértices menor que 1 não é válido porque um grafo deve ter pelo menos um vértice para ser significativo.

## 5.1 Estrutura do Código

Listing 4: Implementação da Main

```
int main() {
    int n;

    cout << "Informe o numero de vertices do grafo completo:-";
    cin >> n;

    if (n < 1) {
        cout << "O numero de vertices deve ser maior que zero." << endl;
        return 1;
    }

    gerarSubgrafos(n);
}
```

```

    return 0;
}

```

## 6 Arquivo Principal

O arquivo principal apresentará o código completo.

### 6.1 Estrutura do Código

Listing 5: all

```

#include <iostream>
#include <vector>
#include <cmath>

using namespace std;

// Função para gerar subgrafos de um grafo completo

void gerarSubgrafos(int n) {
    // Formula para calcular o número de subgrafos de um grafo completo
    int numSubgrafos = pow(2, n * (n - 1) / 2);
    cout << "Número total de subgrafos gerados: " << numSubgrafos << endl;

    vector<pair<int, int>> arestas;

    // Criando a lista de arestas do grafo completo
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            arestas.push_back({i, j});
        }
    }

    // Gerando subgrafos
    for (int mask = 0; mask < numSubgrafos; ++mask) {
        cout << "Subgrafo " << mask + 1 << ":-";
        for (int i = 0; i < arestas.size(); ++i) {
            if (mask & (1 << i)) {
                cout << "(" << arestas[i].first << ", " << arestas[i].second <<
            }
        }
        cout << endl;
    }
}

```

```

int main() {
    int n;

    cout << "Informe o numero de vertices do grafo completo: ";
    cin >> n;

    if (n < 1) {
        cout << "O numero de vertices deve ser maior que zero." << endl;
        return 1;
    }

    gerarSubgrafos(n);

    return 0;
}

```

## Exemplo de Entrada

Aqui está um exemplo de entrada para o nosso programa em C++:

```

// Exemplo de Entrada:
5 //quantidade de vertices que existirão no grafo
1024 //valor total de subgrafos possíveis de acordo com a formula
{0 ... 1024 vezes} //possíveis subgrafos com cada vértice

```

Este exemplo mostra como o programa deve receber os dados de entrada. A primeira linha contém um número inteiro que indica a quantidade de vértices.